

Lista dos prompts utilizados pela equipe dividido por integrantes:

Alex (Tech Lead)

Atividade no Projeto: Coordenação, Modelos (Models), Revisão de Código, Administração do GitHub e Testes Automatizados (JUnit). **Prompts Utilizados:**

- **Contexto e Divisão de Tarefas:** "Ge, irei enviar um arquivo com as orientações de uma nova atividade e preciso que você leia todas com atenção. O tema do projeto do meu grupo será um Sistema de Estacionamento. Quero que você me ajude a coordenar a equipe, garantindo que todos os 7 integrantes participem do desenvolvimento do código."
- **Gestão de Código e Onboarding:** "Crie um texto explicando claramente a tarefa de cada integrante. Em seguida, elabore um guia com os comandos necessários para usar o GitHub (clonar, criar branch, commit, push, merge e pull request), explicando de forma didática e sem jargões por que cada passo é necessário."
- **Cronograma e Dependências:** "Como existem tarefas que dependem da execução de outras, crie um cronograma de ação para cada membro da equipe, deixando claro o que eles já podem ir fazendo e o que depende da entrega de outro colega. Coloque símbolos para indicar o grau de prioridade."
- **Acompanhamento e Entrega:** *(Nota de documentação: Os demais comandos ao longo do projeto foram focados em revisar os códigos desenvolvidos pela equipe, tirar dúvidas pontuais sobre o GitHub e criar o arquivo README de acordo com o que foi pedido na atividade).*

Arielle

Atividade no Projeto: Tarefa 3 - Serviços Básicos + Alerta de Lotação. **Prompts Utilizados:**

- **Contexto da Tarefa:** "Esse é o nosso projeto. Fiquei com a Tarefa 3: Criar `VagaService` e `VeiculoService`, injetar repositórios da Tarefa 1, fazer validações de cadastro e criar lógica de alerta de lotação bloqueando a entrada se atingir 100%."
- **Implementação:** "Com base nos códigos de `VagaRepository` e `VeiculoRepository`, me ajude a implementar somente a minha Tarefa 3, sem alterar a parte dos outros integrantes."
- **Adequação:** "Com base nas classes `Permanencia`, `TipoVeiculo`, `Vaga` e `Veiculo` existentes, ajuste o `VagaService` e `VeiculoService` para usar exatamente os métodos e atributos existentes."
- **Correção de Bug:** "O método `buscarPorPlaca(veiculo.getPlaca())` está vermelho com o erro 'Cannot resolve method'. Como posso corrigir isso sem alterar a tarefa dos meus colegas?"

João

Atividade no Projeto: Tarefa 5 - Endpoints REST (Vaga e Veículo).

Prompts Utilizados:

- "Apresente e explique os comandos básicos do Git essenciais para o fluxo de trabalho habitual no GitHub."
- "Estou realizando uma atividade acadêmica com a seguinte especificação:
TAREFA 5: Endpoints REST (Vaga e Veículo) O que fazer: Criar as classes VagaController e VeiculoController no pacote/pasta controller. Detalhes: Mapear e anotar as classes com @RestController. Rotas: Implementar os endpoints HTTP para testes (ex.: POST para cadastrar, GET para listar), consumindo as regras de negócio dos Services desenvolvidos na Tarefa 3. Com base nesses requisitos, analise e proponha um plano estruturado de implementação."
- "As funções cadastrar e listar estão apresentando os erros exibidos na imagem. Qual é a possível problema e como resolvê-los?"
- "Ao executar comandos no terminal recebo o seguinte erro: 'O termo 'git' não é reconhecido como nome de cmdlet, função, arquivo de script ou programa operável. Verifique a grafia do nome ou, se um caminho tiver sido incluído, veja se o caminho está correto e tente novamente.'Qual é a causa deste problema e quais são os passos para corrigi-lo?"
- "Como realizar a alteração de branch no Git quando os commits ou alterações foram iniciados na branch errada?"

Jonatas

Atividade no Projeto: Tarefa 2 - Repositório de Permanência + Histórico. **Prompts Utilizados:**

- **Prompt 1:** "Diga o que aprendemos até aqui sobre Controller, Service e Repository."
- **Prompt 2:** "Conecte esse conhecimento com o Spring Boot, explicando como a aplicação funcionaria usando conceitos como injeção de dependência e inversão de controle."
- **Prompt 3:** "Tenha em mente o PDF enviado sobre os conhecimentos de back-end, boas práticas e exigências."
- **Prompt 4:** "Preste atenção no PDF sobre o trabalho em grupo (sistema de estacionamento). Analise bem as exigências e como ele funciona, mas não faça nada ainda."
- **Prompt 5:** "[Envio do texto detalhado de divisão de tarefas da equipe]. Analise e não faça nada ainda."

- **Prompt 6:** "Fiquei responsável pela Tarefa 2. Diga o que devo fazer, como fazer, quais exigências existem e o que não posso esquecer. Detalhe atributos, getters/setters, construtores e lógicas dos métodos, levando em consideração os PDFs e a divisão de tarefas."
- **Prompt 7:** "Antes de enviar a classe Permanencia, diga se precisa de mais alguma classe."
- **Prompt 8:** "[Envio das classes auxiliares solicitadas pela IA]."
- **Prompt 9:** "[Envio do código desenvolvido]. Veja o que eu fiz, analise se falta algo de acordo com os PDFs e mande o código corrigido."

Mateus

Atividade no Projeto: Tarefa 4 - Serviço de Permanência + Faturamento. **Prompts Utilizados:**

- **Criação de Lógica:** "Estou com a Tarefa 4. Preciso criar o `PermanenciaService` com: registrar entrada/saída, ocupar/liberar vaga, calcular tempo estacionado, cobrar R\$ 10 (CARRO) e R\$ 5 (MOTO), arredondar hora incompleta e calcular faturamento. Faça validações e use `@Service`, injeção por construtor e `BigDecimal`."
- **Criação de Controller (Apoio):** "Crie um `PermanenciaController` simples com rotas POST, PUT e GET, chamando o `PermanenciaService`. Faça um código completo sem alterar os outros Controllers."
- **Revisão e Correção:** "Revise meu `PermanenciaService` como um trabalho de aluno do 3º período. Confira as validações (vaga ocupada/liberada, tipo de veículo, duplicidade de entrada/saída, cálculo e faturamento). Explique os erros de forma simples e mande o código corrigido."
- **Testes Unitários:** "Crie testes simples com JUnit 5 e Mockito para meu `PermanenciaService`. Teste entradas, saídas, validações, cálculos e faturamento usando `@Mock` e `@InjectMocks`."

Nildson

Atividade no Projeto: Tarefa 6 - Endpoints REST (Permanência, Histórico e Faturamento). **Prompts Utilizados:**

- **Arquitetura e Camadas:** "Como você sugere criar um sistema de estacionamento para desenvolvimento back-end? Com base neste PDF, mostre conceitualmente como seria a interação das camadas (Controller, Service, Repository, Model e Testes), sem gerar código pronto."
- **Estrutura de Classes:** "Com base em um projeto em Java com Spring Boot (entidades Vaga, Veiculo, Permanencia, e camadas em memória), quais as melhores alternativas para estruturar uma classe, seus atributos e métodos?"
- **Responsabilidade (Controller):** "Estou na Tarefa 6 (`PermanenciaController`). Explique qual deve ser a responsabilidade de um

Controller nesta arquitetura e como ele se relaciona com o `PermanenciaService`. Faça 3 perguntas para verificar se entendi."

- **Injeção de Dependência:** "Explique o conceito de injeção de dependência por construtor usando um exemplo genérico. Depois, compare com a estrutura do meu Controller e indique o que precisa ser feito sem reescrever meu código."
- **Parâmetros HTTP:** "Quando devo utilizar `@RequestBody`, `@PathVariable` e `@RequestParam`?"
- **Padrões de Endpoints:** "Para registrar a saída de um veículo, devo usar POST, PUT ou outro? Me dê uma especificação de quatro endpoints essenciais para um estacionamento usando Spring Boot."
- **Revisão de Lógica e Harmonia:** "Olhe o fluxo de saída e verifique se a permanência é atualizada, o valor calculado e a vaga liberada corretamente. Olhe também o meu código e o da minha equipe. Verifique se estão se correlacionando e mostrando harmonia nas camadas."
- **Revisão de Compatibilidade:** "Analise o `VagaService` e `VagaController` e veja se os endpoints estão compatíveis com os métodos do Service."
- **Resolução de Erros:** "Estou com um erro nesta classe. Analise o código, verifique a compatibilidade entre as camadas, explique os problemas de forma simples e proponha apenas alterações necessárias."
- **Git:** "Mostre-me como fazer branch, commit, pull, push e merge no contexto do meu projeto."

Sergio

Atividade no Projeto: Tarefa 1 - Repositórios Básicos (Vaga e Veículo). **Prompts Utilizados:**

- **Análise de Entidades:** "Analise as classes `Vaga.java` e `Veiculo.java` e explique quais informações e atributos serão necessários para construir os Repositories."
- **Estrutura de Pacotes:** "Explique onde deve ser criado o pacote repository e qual será a responsabilidade dessa camada no projeto."
- **Lógica do VagaRepository:** "Explique a lógica para armazenar objetos em memória utilizando uma lista, e como implementar o cadastro, listagem e busca por ID sem apresentar o código completo."
- **Lógica do VeiculoRepository:** "Crie o `VeiculoRepository` usando como referência o `VagaRepository`. Explique quais partes da lógica podem ser reaproveitadas, como buscar por ID e como buscar pela placa (ignorando maiúsculas/minúsculas)."
- **Conceitos do Spring:** "Explique por que as classes devem usar a anotação `@Repository` do Spring e o significado de declarar as listas internas como `final`."
- **Revisão Final:** "Faça uma revisão lógica do `VagaRepository` e `VeiculoRepository` verificando se atendem aos requisitos da Tarefa 1 (armazenamento em memória, cadastro, listagem, buscas, `@Repository` e uso de `final`)."

IA's utilizadas no processo:
Gemini, ChatGPT, Copilot

